

Cahier des Charges Fonctionnel

Projet Fil Rouge CDA - Concepteur Développeur d'Applications

Auteur : BOUGHERARA Safi

Formation : CDA (Concepteur Développeur d'Applications)

Date : Janvier 2026

Version : 2.0 - **RESTAURATION**

Jalon : 1 - Cahier des Charges Fonctionnel

Échéance : 31/01/2026

Table des Matières

1. Contexte Métier
2. Objectifs du Projet
3. Périmètre Fonctionnel
4. Exigences Techniques
5. Contraintes et Enjeux
6. Critères de Succès
7. Annexes

1. Contexte Métier

1.1 Domaine d'Activité

Secteur : Restauration / Hôtellerie-Restauration (CHR)

Cible : Restaurants indépendants, brasseries, bistrots, restaurants gastronomiques

1.2 Problématique Identifiée

Les restaurateurs font face à plusieurs défis dans la gestion de leurs réservations :

Problèmes Actuels

- **Charge téléphonique élevée** : Le personnel en salle doit gérer les appels de réservation pendant le service, perturbant l'expérience client
- **Disponibilité limitée** : Impossibilité de prendre des réservations en dehors des heures d'ouverture (matin, après-service, jours de fermeture)
- **Perte de clients** : Les clients qui n'obtiennent pas de réponse immédiate réservent chez un concurrent
- **Erreurs de réservation** : Risques de double réservation, mauvaise estimation du nombre de couverts, oubli de rappel

- **Gestion complexe des tables** : Difficulté à optimiser le remplissage (tables de 2, 4, 6, 8 personnes)
- **No-show** : Clients qui ne viennent pas sans prévenir, entraînant une perte de chiffre d'affaires
- **Coût du personnel** : Nécessité d'avoir du personnel dédié à la prise de réservations

1.3 Solution Proposée : CALENDRIA

CALENDRIA est un **assistant de réservation virtuel basé sur l'intelligence artificielle**, conçu pour automatiser la gestion des réservations de tables dans les restaurants.

Fonctionnement Global

1. **Client initie la réservation** : Via l'un des 3 canaux disponibles :
 -  **WhatsApp Business** : Scan QR Code ou lien direct
 -  **Widget Web** : Chatbot intégré sur le site du restaurant
 -  **SMS** : Message vers le numéro dédié
2. **Conversation IA** : Le chatbot engage une conversation naturelle et collecte :
 - Nom du client
 - Numéro de téléphone (si pas déjà connu)
 - Nombre de personnes (couverts)
 - Date et heure souhaitées
 - Demandes spéciales (allergie, anniversaire, etc.) - optionnel
3. **Vérification de disponibilité en temps réel** :
 - Consultation des tables disponibles selon leur capacité
 - Vérification des créneaux horaires configurés
 - Calcul du nombre de couverts déjà réservés
 - Application de l'algorithme d'attribution intelligent
4. **Validation automatique** :
 -  **Si disponible** : Réservation confirmée instantanément
 -  **Si indisponible** : Proposition automatique de créneaux alternatifs
5. **Confirmation multi-canal** :
 - Confirmation immédiate dans le canal utilisé (WhatsApp/Widget/SMS)
 - SMS de confirmation avec détails complets
6. **Dashboard restaurateur** :
 - Visualisation en temps réel de la nouvelle réservation
 - Notification push (optionnel)
7. **Rappel automatique** :
 - SMS 24h avant avec demande de confirmation
 - Réduction des no-show

Différence Clé avec le Secteur Médical

Validation automatique : Contrairement au secteur médical où le praticien doit valider chaque RDV, ici la réservation est **confirmée automatiquement** si des tables sont disponibles. Le restaurateur n'intervient que pour consulter, modifier ou annuler.

1.4 Utilisateurs Cibles

Utilisateurs Principaux

1. **Restaurateurs / Gérants**

- Configurent les paramètres du restaurant (tables, horaires, services)
- Consultent les réservations du jour/semaine
- Peuvent annuler ou modifier une réservation
- Analysent les statistiques de remplissage

2. **Clients**

- Réservent via WhatsApp, Widget Web ou SMS
- Reçoivent des confirmations instantanées
- Peuvent annuler ou modifier via le même canal

Utilisateurs Secondaires

1. **Personnel de salle** (optionnel)

- Consultent les réservations du jour
- Marquent les clients comme "arrivés" ou "no-show"
- Gèrent les cas particuliers

1.5 Valeur Ajoutée

- **Disponibilité 24/7** : Les clients peuvent réserver à tout moment, même à 2h du matin
- **Gain de temps** : Réduction de 70% du temps consacré à la gestion des appels
- **Réduction des erreurs** : Automatisation de la gestion des tables et couverts
- **Amélioration de l'expérience client** : Réponse immédiate, pas d'attente
- **Optimisation du remplissage** : Algorithme intelligent pour maximiser l'occupation des tables
- **Réduction des no-show** : Rappels automatiques et demande de confirmation
- **Augmentation du chiffre d'affaires** : Plus de réservations captées, meilleur taux de remplissage

2. Objectifs du Projet

Les objectifs de CALENDRIA sont définis selon la méthode **SMART** :

2.1 Objectifs SMART

Critère	Description
Spécifique	Développer un assistant virtuel multi-canal capable de gérer automatiquement 90% des réservations de tables pour un restaurant via WhatsApp, Widget Web et SMS
Mesurable	- Réduire le temps de gestion des réservations de 70% - Taux de complétion chatbot > 95% (réaliste avec texte structuré) - Validation automatique instantanée si disponibilité - Au minimum 2 canaux fonctionnels (WhatsApp + Widget ou SMS) - Réduction des no-show de 30% grâce aux rappels
Atteignable	Utilisation de technologies éprouvées (Twilio Conversations/SMS, OpenAI GPT-4o-mini, Symfony, React) avec un périmètre fonctionnel réaliste pour 6 mois
Réaliste	MVP chatbot fonctionnel pour 1 restaurant, développement séquentiel (WhatsApp → Widget → SMS), possibilité d'extension vocale en bonus
Temporellement défini	Livraison finale : 30 juin 2026 (6 mois de développement avec jalons mensuels)

2.2 Objectifs Fonctionnels

1. **Automatiser les réservations** : Permettre aux clients de réserver une table sans intervention humaine
2. **Optimiser le remplissage** : Maximiser l'occupation des tables en proposant les créneaux optimaux
3. **Centraliser les réservations** : Regrouper toutes les réservations dans une interface unique
4. **Réduire les no-show** : Système de rappels et confirmations automatiques
5. **Simplifier la gestion** : Interface intuitive pour le restaurateur

2.3 Objectifs Techniques

1. **Respecter le cahier des charges technique** : Utiliser la stack imposée (Symfony, React, SQL, Docker, CI/CD)
2. **Garantir la sécurité** : Conformité RGPD (données personnelles simples, pas de données de santé)
3. **Assurer la qualité** : Couverture de tests > 80%, respect des normes OWASP
4. **Faciliter le déploiement** : Application conteneurisée et déployable en 1 commande

3. Périmètre Fonctionnel

3.1 Fonctionnalités Principales (IN SCOPE)

3.1.1 Module IA Conversationnel - Multi-Canal

Description : Système de réservation intelligent accessible via 3 canaux complémentaires

Architecture Partagée :

- Tous les canaux utilisent la même API REST Symfony
- Même logique de chatbot (OpenAI GPT-4o-mini)
- Même algorithme de disponibilité
- Réponses adaptées au format de chaque canal

Canal 1 : WhatsApp Business (Priorité 1 - Obligatoire)

Description : Chatbot conversationnel via WhatsApp

Accès client :

- QR Code affiché sur les menus du restaurant
- Lien partageable sur réseaux sociaux / site web
- Envoi par SMS : "Réservez via WhatsApp : [lien]"

Fonctionnalités :

- ☒ Conversation naturelle en français
- ☒ Collecte d'informations :
 - Nom du client
 - Numéro de téléphone (automatique via WhatsApp)
 - Nombre de personnes (couverts)
 - Date et heure souhaitées
 - Demandes spéciales (allergie, anniversaire, etc.)
- ☒ Vérification de disponibilité en temps réel
- ☒ Validation automatique si disponible
- ☒ Proposition de créneaux alternatifs si indisponible
- ☒ Confirmation immédiate dans WhatsApp
- ☒ Historique de conversation conservé
- ☒ Notifications push natives

Exemple de conversation :

Bot: Bonjour ! Je suis l'assistant de [Nom Restaurant].

Pour quelle date souhaitez-vous réserver ?

Client: Demain soir

Bot: Parfait ! Pour combien de personnes ?

Client: 4 personnes

Bot: À quelle heure préférez-vous ?

Client: 20h

Bot:  Table pour 4 disponible demain 10/01 à 20h00.

Puis-je avoir votre nom ?

Client: Dupont

Bot: 🎉 Réservez confirmée !

 10/01/2026 à 20h00

 4 personnes

 [Adresse restaurant]

Vous recevrez un SMS de rappel 24h avant.

Technologies :

- Twilio Conversations API (WhatsApp Business)
- OpenAI GPT-4o-mini (génération de réponses)
- Webhooks pour réception/envoi de messages

Coûts :

- Développement : Gratuit (Twilio Sandbox)
- Production : ~0,005\$/message (~5€/mois pour 100 réservations)

Canal 2 : Widget Web (Priorité 2 - Fortement Recommandé)

Description : Chatbot intégrable sur le site du restaurant

Intégration :

<!-- Code à copier-coller sur le site du restaurant -->

```
<script src="https://app.calendria.fr/widget.js"></script>
```

```
<div id="calendria-chatbot"
```

```
  data-restaurant-id="abc123"
```

```
  data-theme="light">
```

```
</div>
```

Rendu :

- Bulle de chat en bas à droite du site (comme Intercom, Crisp)
- Clic → Ouverture du chatbot
- Design personnalisable (couleurs du restaurant)

Fonctionnalités :

- ☒ Même conversation que WhatsApp
- ☒ Interface responsive (desktop + mobile)
- ☒ Sélecteurs visuels (date picker, nombre de personnes)
- ☒ Confirmation instantanée
- ☒ Pas d'application à installer

Technologies :

- React component (iframe)
- API REST Symfony
- OpenAI GPT-4o-mini

Coûts :

- Développement : Inclus dans le projet
- Production : Gratuit (hébergé avec l'API)

Description : Réservation par SMS vers numéro dédié

Accès client :

- Affichage sur menus : "Réservez par SMS au 06 XX XX XX XX"
- Client envoie : "Bonjour, je voudrais réserver pour 4 demain soir à 20h"

Fonctionnalités :

- ☒ Conversation guidée par SMS
- ☒ Même logique que WhatsApp (format texte court)
- ☒ Confirmation par SMS
- ☒ Utile pour clients sans WhatsApp

Limite :

- Moins conversationnel (coût par SMS)
- Mieux adapté pour confirmations/rappels que prise de réservation complète

Technologies :

- Twilio SMS API
- Webhooks pour réception/envoi

Coûts :

- Production : ~0,0075\$/SMS (~7€/mois pour 100 réservations)

Stratégie de Développement Phasée :

Phase 1 (Mars) : API + Algorithme

- ☒ API REST Symfony
- ☒ Algorithme de disponibilité
- ☒ Tests avec Postman

Phase 2 (Avril) : WhatsApp + Dashboard

- ☒ Chatbot WhatsApp (canal principal)
- ☒ Dashboard restaurateur
- ☒ SMS de confirmation

Phase 3 (Mai) : Widget Web + SMS

- ☒ Widget Web (React component)
- ☒ SMS direct (réutilise logique WhatsApp)
- ☒ Tests + Sécurité

Phase 4 (Juin) : Finalisation

- ☒ Déploiement Docker
- ☒ Documentation
- ☒ (Bonus optionnel : Voix téléphonique si temps restant)

Critères de Succès :

- ☒ Au minimum 2 canaux fonctionnels (WhatsApp + Widget ou WhatsApp + SMS)
- ☒ Taux de complétion > 95% (réaliste avec texte structuré)
- ☒ Temps moyen de conversation < 2 minutes
- ☒ Validation automatique instantanée si disponibilité

3.1.2 Algorithme de Gestion des Tables

Description : Logique métier pour optimiser l'attribution des tables

Fonctionnalités :

A. Configuration des Tables

- ☒ Définition des tables du restaurant :
- Numéro de table
- Capacité (nombre de places)
- Type (intérieur, terrasse, VIP, etc.)
- Statut (disponible, hors service)
- ☒ Exemple : Table 1 (2 places), Table 2 (4 places), Table 3 (6 places), etc.

B. Gestion des Services

- ☒ Configuration des créneaux horaires :
- **Service midi** : 12h00 - 14h30
- **Service soir** : 19h00 - 22h30
- ☒ Durée moyenne d'un repas (configurable) :
- Midi : 1h30
- Soir : 2h00
- ☒ Créneaux de réservation (par tranche de 15 ou 30 minutes)

C. Algorithme d'Attribution

- ☒ **Règles d'attribution intelligente :**
- 1. Privilégier les tables de capacité exacte (2 personnes → table de 2)
- 2. Si pas de table exacte, prendre la table immédiatement supérieure (2 personnes → table de 4 si pas de table de 2)
- 3. Éviter de "gaspiller" les grandes tables pour peu de personnes (sauf si nécessaire)
- 4. Vérifier que la table sera libérée à temps pour le créneau demandé
- ☒ **Calcul de disponibilité :**

Exemple : Restaurant avec 10 tables (4x2 places, 4x4 places, 2x6 places)

Capacité totale : 40 couverts

Demande : 4 personnes à 20h00

→ Vérifier si une table de 4 est libre à 20h00

→ Si oui : Réserver automatiquement

→ Si non : Proposer 19h30 ou 20h30 ou 21h00

D. Gestion des Contraintes

- ☒ Nombre maximum de couverts par service (configurable)
- ☒ Délai minimum de réservation (ex: 2h à l'avance)
- ☒ Délai maximum de réservation (ex: 30 jours à l'avance)
- ☒ Blocage de dates spécifiques (fermeture exceptionnelle, événement privé)

3.1.3 Interface Restaurateur (Dashboard Web)

Description : Application web pour la gestion des réservations par le restaurateur

Fonctionnalités :

A. Authentification et Sécurité

- ☒ Page de connexion sécurisée (email + mot de passe)
- ☒ Hashage des mots de passe (bcrypt/Argon2)
- ☒ Gestion de session sécurisée
- ☒ Déconnexion automatique après inactivité
- ☒ Réinitialisation de mot de passe par email

B. Tableau de Bord (Dashboard)

- ☒ Vue d'ensemble du jour :
- Nombre de réservations confirmées
- Nombre de couverts attendus
- Taux de remplissage (% de tables occupées)
- Prochaines arrivées
- ☒ Statistiques en temps réel
- ☒ Alertes (réservations dans les 30 prochaines minutes)

C. Gestion du Calendrier

- ☒ Visualisation du planning :
- Vue jour (liste chronologique des réservations)
- Vue semaine
- Vue mois
- ☒ Affichage des réservations confirmées
- ☒ Affichage des créneaux disponibles
- ☒ Code couleur par statut :

-  Confirmée
-  Arrivée imminente (< 30 min)
-  Client arrivé
-  No-show
-  Annulée
-  Filtres par service (midi/soir), date, nombre de couverts

D. Gestion des Réservations

-  Liste de toutes les réservations
-  Détails de chaque réservation :
- Nom du client
- Téléphone
- Nombre de personnes
- Date et heure
- Table attribuée
- Demandes spéciales
- Statut
-  Actions possibles :
- **Modifier** : Changer l'heure, le nombre de personnes, la table
- **Annuler** : Annuler la réservation et envoyer SMS au client
- **Marquer comme arrivé** : Confirmer la présence du client
- **Marquer comme no-show** : Client absent sans prévenir
-  Recherche de réservations (nom, téléphone, date)

E. Gestion des Tables

-  Configuration des tables :
- Ajouter/Supprimer/Modifier une table
- Numéro, capacité, type
- Mettre hors service temporairement
-  Plan de salle visuel (optionnel, bonus)
-  Statistiques par table (taux d'occupation)

F. Gestion des Horaires et Services

-  Configuration des horaires d'ouverture par jour de la semaine
-  Définition des services (midi, soir)
-  Durée moyenne des repas
-  Créneaux de réservation (15 ou 30 minutes)
-  Nombre maximum de couverts par service
-  Gestion des fermetures exceptionnelles

G. Gestion des Clients

-  Liste des clients enregistrés
-  Fiche client :
- Informations personnelles
- Historique des réservations

- Taux de no-show
- Notes du restaurateur (client VIP, allergies, préférences)
- ☒ Recherche de clients (nom, téléphone)
- ☒ Export des données client (conformité RGPD)

H. Statistiques et Rapports

- ☒ Statistiques de remplissage :
- Taux d'occupation par jour/semaine/mois
- Nombre de couverts servis
- Créneaux les plus demandés
- ☒ Analyse des no-show :
- Taux de no-show global
- Clients avec no-show récurrents
- ☒ Export des données (CSV, PDF)

I. Paramètres

- ☒ Modification des informations du restaurant
- ☒ Configuration des notifications
- ☒ Gestion du consentement RGPD
- ☒ Politique de confidentialité
- ☒ Personnalisation des messages de l'IA

Technologies :

- React 18+ (avec TypeScript recommandé)
- React Router (navigation)
- Axios (appels API)
- Material-UI ou Tailwind CSS (design)
- React Query (gestion du cache)
- FullCalendar (composant calendrier)

3.1.4 Système de Notifications SMS (API Twilio)

Description : Envoi automatique de SMS aux clients

Fonctionnalités :

- ☒ **SMS de confirmation immédiate** après réservation automatique
- ☒ **SMS de proposition alternative** si créneau indisponible
- ☒ **SMS de rappel 24h avant** la réservation
- ☒ **SMS de demande de confirmation** 24h avant (pour réduire no-show)
- ☒ **SMS d'annulation** si le restaurateur annule
- ☒ Gestion des réponses SMS du client (confirmation, annulation)

Format des SMS :

[CONFIRMATION AUTOMATIQUE]

Bonjour [Nom],

Votre réservation au [Nom Restaurant] est confirmée :

 [Date] à [Heure]

 [Nombre] personnes

 [Adresse]

Pour annuler, appelez le [Téléphone]

[CRÉNEAU INDISPONIBLE]

Bonjour [Nom],

Le créneau demandé ([Date] à [Heure]) est complet.

Créneaux disponibles :

- [Date] à [Heure1]

- [Date] à [Heure2]

Répondez 1 ou 2 pour confirmer.

[RAPPEL 24H]

Rappel : Réservation demain [Date] à [Heure]

[Nom Restaurant] - [Nombre] personnes

Répondez OUI pour confirmer ou NON pour annuler.

[ANNULATION]

Votre réservation du [Date] à [Heure] au [Nom Restaurant] a été annulée.

Nous espérons vous revoir bientôt !

Technologies :

- Twilio SMS API
- Gestion des webhooks pour les réponses SMS

3.1.5 Gestion des Clients (Base de Données)

Description : Stockage et gestion des données clients

Fonctionnalités :

-  Création automatique de fiche client lors du premier appel
-  Mise à jour des informations client
-  Historique complet des réservations (passées, à venir, annulées, no-show)
-  Consentement RGPD enregistré
-  Suppression de compte (droit à l'oubli)
-  Anonymisation des données après suppression

Données stockées :

- Nom
- Numéro de téléphone (obligatoire)
- Email (optionnel)
- Préférences (allergies, demandes spéciales)
- Consentement RGPD (obligatoire)
- Date de création du compte
- Taux de no-show

Conformité RGPD :

-  **Données personnelles simples** (pas de données de santé sensibles)
- Consentement explicite lors du premier appel
- Possibilité d'export des données
- Suppression de compte possible
- Durée de conservation : 2 ans après dernière réservation

3.2 Fonctionnalités Hors Scope (OUT OF SCOPE)

Les fonctionnalités suivantes ne seront **PAS** développées dans le cadre de ce projet (possibles évolutions futures) :

-  **Paiement en ligne** : Pas de gestion des acomptes ou paiements
-  **Commande en ligne** : Pas de menu en ligne ou commande de plats
-  **Gestion des stocks** : Pas de gestion de la cuisine ou des stocks
-  **Application mobile native** : Uniquement interface web responsive
-  **Multi-langues** : Français uniquement
-  **Multi-restaurants** : MVP pour un seul restaurant (extension future possible)
-  **Intégration avec logiciels de caisse** : Pas de connexion avec TPV existants
-  **Système de fidélité** : Pas de programme de points ou récompenses
-  **Avis clients** : Pas de système de notation ou reviews
-  **Gestion du personnel** : Pas de planning des employés

4. Exigences Techniques

4.1 Architecture Retenue

Choix : Architecture API REST Symfony + Front-end React (SPA)

Justification du Choix

Critère	Architecture Monolithique	Architecture API REST + SPA	Choix
Séparation des responsabilités	Moyenne (Twig + Symfony)	Excellente (back/front séparés)	✅ API REST
Scalabilité	Limitée	Élevée (possibilité d'ajouter app mobile)	✅ API REST
Expérience utilisateur	Rechargement de pages	Interface dynamique temps réel	✅ API REST
Complexité de développement	Faible	Moyenne	⚠️ Acceptable
Intégration Twilio	Possible	Nécessaire (webhooks API)	✅ API REST
Évolutivité	Difficile	Facile (ajout de clients)	✅ API REST

Conclusion : L'architecture **API REST + React** est retenue car :

1. **Nécessité d'une API** : Twilio communique via webhooks HTTP, une API REST est donc indispensable
2. **Interface temps réel** : Le dashboard restaurateur doit afficher les nouvelles réservations en temps réel
3. **Évolutivité** : Possibilité future d'ajouter une application mobile pour les clients
4. **Séparation des préoccupations** : Back-end (logique métier complexe de gestion des tables) et front-end (présentation) totalement découplés

4.2 Stack Technique Détaillée

Back-end

Composant	Technologie	Version	Justification
Framework	Symfony	6.4 LTS	Framework PHP imposé, version LTS pour stabilité
Langage	PHP	8.2+	Dernière version stable, typage strict
ORM	Doctrine	2.x	Intégré à Symfony, prévention injections SQL
API Platform	API Platform	3.x	Génération automatique d'API REST, documentation OpenAPI
Authentication	LexikJWTAuthenticationBundle	2.x	Tokens JWT pour API stateless
Validation	Symfony Validator	6.x	Validation des données en entrée
Sérialisation	Symfony Serializer	6.x	Transformation entités ↔ JSON

Front-end

Composant	Technologie	Version	Justification
Framework	React	18+	Framework JavaScript imposé, écosystème riche
Langage	TypeScript	5.x	Typage statique, meilleure maintenabilité
Routage	React Router	6.x	Navigation SPA
État global	React Context / Zustand	-	Gestion d'état simple
Requêtes HTTP	Axios	1.x	Client HTTP avec intercepteurs
UI Framework	Material-UI (MUI)	5.x	Composants React prêts à l'emploi, responsive
Formulaires	React Hook Form	7.x	Gestion performante des formulaires
Calendrier	FullCalendar	6.x	Composant calendrier professionnel

Base de Données

Composant	Technologie	Version	Justification
SGBD	PostgreSQL ou MySQL	15+ / 8+	À décider : PostgreSQL recommandé pour contraintes avancées
Modélisation	Méthode MERISE	-	Imposée par le CDC (MCD/MLD/MPD)
Migrations	Doctrine Migrations	3.x	Versionnement du schéma de base de données
Normalisation	3NF	-	Forme normale 3 pour éviter redondances

Choix PostgreSQL vs MySQL :

- **PostgreSQL** : Meilleure gestion des contraintes, types avancés (JSON pour demandes spéciales), performances sur requêtes complexes → **Recommandé**
- **MySQL** : Plus simple, écosystème plus large

APIs Externes

API	Fournisseur	Usage	Coût
Téléphonie	Twilio Voice API	Réception d'appels, reconnaissance vocale	15,50\$ crédit gratuit, puis ~0,013\$/min
SMS	Twilio SMS API	Envoi de notifications	Inclus dans crédit gratuit, puis ~0,0075\$/SMS
IA Conversationnelle	OpenAI GPT-4 API	Génération de réponses naturelles	Crédit gratuit limité, puis ~0,03\$/1K tokens

Stratégie de gestion des coûts :

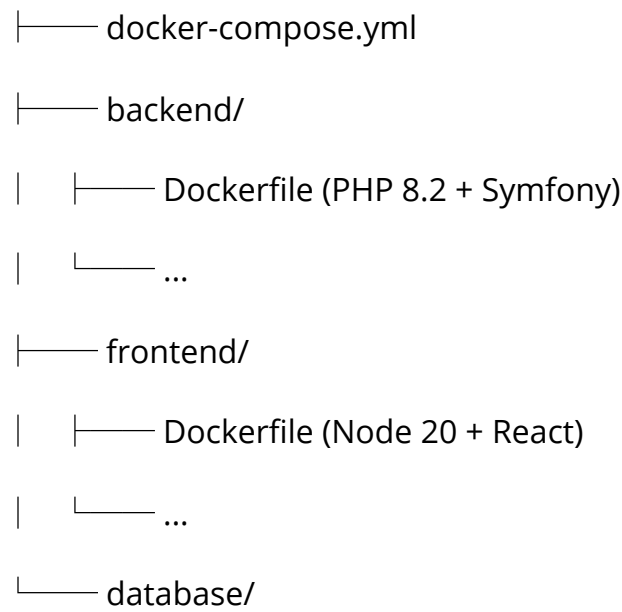
- 1. Utiliser les **crédits gratuits** pour le développement et les tests
- 2. Utiliser les **numéros de test Twilio** pour simuler sans consommer de crédit
- 3. **Limiter les appels** : Tester d'abord avec chatbot texte (gratuit)
- 4. **Surveiller l'usage** : Dashboard Twilio/OpenAI pour monitoring
- 5. **Passer en production payante** uniquement après validation complète

Conteneurisation

Composant	Technologie	Version	Justification
Conteneurs	Docker	24+	Imposé par le CDC, environnement uniforme
Orchestration	Docker Compose	2.x	Gestion multi-conteneurs (API, BDD, Front)
Images de base	php:8.2-fpm, postgres:15, node:20	-	Images officielles optimisées

Architecture Docker :

calendria/



└─── init.sql (données initiales)

Contrôle de Version et CI/CD

Composant	Technologie	Justification
Versioning	Git	Imposé par le CDC
Hébergement	GitHub	Écosystème complet (Actions, Projects, Wiki)
CI	GitHub Actions	Intégré à GitHub, gratuit pour projets publics
Tests automatisés	PHPUnit, Behat, Jest	Tests unitaires + fonctionnels + front
Qualité du code	PHPStan, ESLint	Analyse statique PHP/TypeScript
Déploiement	Docker Hub	Stockage des images Docker

Stratégie de branches Git :

main → Production (releases jalons)
develop → Intégration continue
feature/* → Développement de fonctionnalités
hotfix/* → Corrections urgentes

Pipeline CI/CD :

```
# .github/workflows/ci.yml  
on: [push, pull_request]  
  
jobs:  
  
  tests:  
  
    - Linter PHP (PHPStan)
```

- Tests unitaires (PHPUnit)
- Tests fonctionnels (Behat)
- Linter TypeScript (ESLint)
- Tests front (Jest)

build:

- Build image Docker backend
- Build image Docker frontend

deploy:

- Push sur Docker Hub (si tag release)

Tests

Type de Test	Outil	Couverture Cible
Tests unitaires back	PHPUnit	> 80% des classes métier
Tests fonctionnels back	Behat	Tous les cas d'usage principaux
Tests API	Postman/Newman	Tous les endpoints
Tests unitaires front	Jest + React Testing Library	> 70% des composants
Tests E2E	Cypress (optionnel)	Parcours utilisateur complets

4.3 Sécurité et Conformité

Sécurité OWASP Top 10

Menace	Protection Implémentée
Injection SQL	ORM Doctrine avec requêtes préparées, validation des entrées
XSS	Échappement automatique React, Content Security Policy
CSRF	Tokens JWT (API stateless), SameSite cookies
Authentification	Hashage bcrypt/Argon2, politique de mot de passe forte
Exposition de données	HTTPS obligatoire en production
Contrôle d'accès	Rôles Symfony (ROLE_RESTAURATEUR, ROLE_ADMIN)
Configuration	Variables d'environnement (.env), pas de secrets en dur
Désérialisation	Validation stricte des données JSON
Logging	Logs d'accès aux données clients (audit RGPD)
Monitoring	Surveillance des erreurs (Sentry optionnel)

Conformité RGPD

✅ **Simplifié** : CALENDRIA manipule des **données personnelles simples** (nom, téléphone, préférences alimentaires), **PAS de données de santé sensibles**.

Mesures obligatoires :

1. **Consentement explicite** :

- Information lors du premier appel
- Enregistrement de la date de consentement
- Possibilité de retirer le consentement

2. **Droit à l'information** :

- Politique de confidentialité accessible

- Mention des finalités de traitement (gestion des réservations)
 - Durée de conservation des données (2 ans)
3. **Droit d'accès :**
- Export des données client en format lisible (JSON/PDF)
4. **Droit à l'oubli :**
- Suppression complète du compte client
 - Anonymisation des réservations passées (pour statistiques restaurant)
5. **Sécurité des données :**
- HTTPS obligatoire en production
 - Logs d'accès aux données
6. **Durée de conservation :**
- Données clients : 2 ans après dernière réservation
 - Logs : 1 an maximum

Implémentation technique :

// Exemple : Gestion du consentement RGPD

```
class Client {

    #[ORM\Column(type: 'boolean')]

    private bool $consentementRgpd = false;


    #[ORM\Column(type: 'datetime', nullable: true)]

    private ?DateTime $dateConsentement = null;


    public function donnerConsentement(): void {

        $this->consentementRgpd = true;

        $this->dateConsentement = new DateTime();

    }

}
```

4.4 Contraintes Techniques Imposées

✓ Respectées :

- Back-end Symfony (PHP 8+)
- Front-end React (SPA)
- Base de données SQL (PostgreSQL ou MySQL)
- API externe (Twilio + OpenAI)

- Docker + Docker Compose
- Git + CI/CD (GitHub Actions)
- Tests automatisés (PHPUnit + Behat + Jest)
- Sécurité OWASP + RGPD

5. Contraintes et Enjeux

5.1 Contraintes Temporelles

Durée totale : 6 mois (janvier à juin 2026)

Jalons mensuels :

Jalon	Mois	Échéance	Livrable
1	Janvier	31/01/2026	CDCF (ce document)
2	Février	28/02/2026	Méthodologie + Maquettes UI/UX
3	Mars	31/03/2026	MCD/MLD/MPD + API REST + Algorithme tables
4	Avril	30/04/2026	Chatbot WhatsApp + Dashboard restaurateur + UML
5	Mai	29/05/2026	Widget Web + SMS + Tests + Sécurité
6	Juin	30/06/2026	Finalisation + Déploiement + Documentation + (Bonus : Voix)

Charge de travail estimée : ~20h/semaine

5.2 Contraintes Réglementaires

RGPD (Règlement Général sur la Protection des Données)

-  Consentement explicite pour collecte de données
-  Droit d'accès, rectification, suppression
-  Durée de conservation limitée (2 ans)
-  Sécurité des données (HTTPS)
-  Notification en cas de fuite de données (< 72h)

Note : Pas de données de santé sensibles, donc RGPD simplifié par rapport au secteur médical.

5.3 Contraintes Budgétaires

APIs Externes :

Canal/API	Crédit Gratuit	Coût Production (100 résas/mois)
WhatsApp (Twilio Conversations)	Sandbox gratuit	~5€ (0,005\$/message)
Widget Web	N/A (hébergé avec API)	Gratuit (inclus)
SMS (Twilio SMS)	Inclus dans crédit	~7€ (0,0075\$/SMS)
OpenAI GPT-4o-mini	5\$ crédit initial	~5€ (0,002\$/1K tokens)
TOTAL	Gratuit en dev	~15-20€/mois

Stratégie :

- Utiliser Twilio Sandbox WhatsApp pour développement (gratuit)
- Tester avec numéros de test Twilio (gratuit)
- OpenAI GPT-4o-mini (10x moins cher que GPT-4)
- Mocks locaux pour tests unitaires
- Prévoir budget ~20€/mois en production

5.4 Risques Identifiés et Mitigation

Risque	Probabilité	Impact	Mitigation
Dépassement crédit API	Élevée Faible	Faible	Chatbot 10x moins cher que vocal, mocks locaux, surveillance usage
Complexité algorithme tables	Moyenne	Élevé	Mock 1000 scénarios Python, buffer turnover configurable, tests exhaustifs
Complexité Widget iframe	Faible	Faible	Commencer par version simple, pas de personnalisation avancée
Retard sur planning	Faible	Moyen	Développement dès Mars, Widget/SMS optionnels si retard
Bugs en production	Moyenne	Moyen	Tests automatisés > 80%, tests manuels avant chaque jalon
Indisponibilité API Twilio/OpenAI	Faible	Élevé	Gestion d'erreurs robuste, logs détaillés, fallback gracieux

5.5 Hypothèses

-  Les APIs Twilio et OpenAI restent accessibles en version gratuite/test pendant le développement
-  Un seul restaurant pour le MVP (extension multi-restaurants en V2)
-  Clients francophones uniquement (pas de multi-langues)
-  Accès à un environnement de test (serveur ou local) pour déploiement Docker
-  Pas de données réelles de clients pendant le développement (données fictives)
-  Restaurant type : 10-20 tables, 2 services (midi/soir)

6. Critères de Succès

6.1 Critères Fonctionnels

✅ **Le projet est considéré comme réussi si :**

1. **Chatbot Multi-Canal :**

- Au minimum **2 canaux fonctionnels** (WhatsApp + Widget ou WhatsApp + SMS)
- Taux de complétion > **95%** (réaliste avec texte structuré)
- Collecte complète des informations (nom, téléphone, nombre de personnes, date/heure) dans **95%** des cas
- Temps moyen de conversation < **2 minutes**

2. **Algorithme de Disponibilité :**

- Calcul correct de la disponibilité dans **100%** des cas
- Proposition automatique de créneaux alternatifs si indisponible
- Optimisation de l'attribution des tables (pas de gaspillage)
- Temps de calcul < **200ms**

3. **Validation Automatique :**

- **100%** des réservations disponibles sont confirmées automatiquement
- Confirmation envoyée en < **10 secondes** (WhatsApp/Widget/SMS)

4. **Interface Restaurateur :**

- Le restaurateur peut consulter toutes les réservations du jour en < **5 secondes**
- Modification/Annulation d'une réservation en < **30 secondes**
- Statistiques de remplissage affichées correctement
- Interface responsive (desktop + mobile)

5. **Réduction des No-Show :**

- Système de rappels 24h avant opérationnel
- Taux de réponse aux rappels > **70%**

6.2 Critères Techniques

✅ **Exigences techniques respectées :**

1. **Qualité du Code :**

- Couverture de tests > **80%** (back-end)
- Couverture de tests > **70%** (front-end)
- Respect des normes PSR-12 (PHP)
- Respect des conventions ESLint (TypeScript)

2. **Sécurité :**

- **100%** des tests de sécurité OWASP passent
- Aucune vulnérabilité critique détectée
- Conformité RGPD validée (checklist complète)

3. **Performance :**

- Temps de chargement de l'interface < 2 secondes
- Temps de réponse API < 500ms (95e percentile)
- Calcul de disponibilité < 200ms
- Application utilisable sur mobile (responsive)

4. **Déploiement :**

- Application déployable via Docker en **1 commande** (docker-compose up)
- Pipeline CI/CD fonctionnelle (tests automatiques à chaque push)
- Documentation d'installation complète et claire

6.3 Critères de Présentation

✅ **Soutenance réussie si :**

1. **Démonstration :**

- Démonstration live fonctionnelle (ou vidéo de secours)
- Parcours complet : appel → validation automatique → SMS → consultation dashboard

2. **Documentation :**

- Rapport final complet (tous les chapitres)
- Diagrammes UML clairs et cohérents
- Code commenté et lisible

3. **Compréhension :**

- Capacité à expliquer l'architecture
- Justification des choix techniques (notamment algorithme de tables)
- Réponses pertinentes aux questions du jury

7. Annexes

7.1 Glossaire

Terme	Définition
API REST	Architecture d'API basée sur HTTP et les verbes REST (GET, POST, PUT, DELETE)
CDCF	Cahier des Charges Fonctionnel (ce document)
CDCT	Cahier des Charges Technique (fourni par la formation)
CI/CD	Continuous Integration / Continuous Deployment (intégration et déploiement continus)
Couverts	Nombre de personnes pour une réservation
Doctrine	ORM (Object-Relational Mapping) pour PHP/Symfony
JWT	JSON Web Token (système d'authentification par tokens)
MCD/MLD/MPD	Modèle Conceptuel/Logique/Physique de Données (méthode MERISE)
MVP	Minimum Viable Product (produit minimum viable)
No-show	Client qui ne se présente pas à sa réservation sans prévenir
ORM	Object-Relational Mapping (mapping objet-relationnel)
OWASP	Open Web Application Security Project (référentiel de sécurité web)
RGPD	Règlement Général sur la Protection des Données

Service	Période de service (midi ou soir)
SPA	Single Page Application (application web monopage)
Twilio	Plateforme cloud pour téléphonie et SMS
Webhook	URL appelée automatiquement par un service externe lors d'un événement

7.2 Modèle de Données (Aperçu)

Entités Principales

1. **RESTAURANT**

- id_restaurant (PK)
- nom
- adresse
- telephone
- email
- capacite_totale (nombre de couverts max)

2. **TABLE**

- id_table (PK)
- id_restaurant (FK → RESTAURANT)
- numero_table
- capacite (nombre de places)
- type (intérieur, terrasse, VIP)
- statut (disponible, hors_service)

3. **SERVICE**

- id_service (PK)
- id_restaurant (FK → RESTAURANT)
- type (midi, soir)
- heure_debut
- heure_fin
- duree_moyenne_repas (minutes)

4. **CLIENT**

- id_client (PK)
- nom
- telephone (UNIQUE)
- email (NULLABLE)
- preferences (JSON : allergies, demandes spéciales)
- consentement_rgpd (BOOLEAN)
- date_consentement
- taux_noshow (calculé)

5. **RESERVATION**

- id_reservation (PK)
- id_client (FK → CLIENT)
- id_restaurant (FK → RESTAURANT)
- id_table (FK → TABLE, NULLABLE initialement)
- date_reservation
- heure_reservation
- nombre_personnes
- demandes_speciales (TEXT)
- statut (ENUM: 'confirmée', 'annulée', 'noshow', 'terminée')
- date_creation
- date_modification

6. **CRENEAU_HORAIRE**

- id_creneau (PK)
- id_restaurant (FK → RESTAURANT)
- jour_semaine (ENUM: 'lundi', 'mardi', ...)
- heure_debut
- heure_fin
- actif (BOOLEAN)

7. **CONVERSATION_IA**

- id_conversation (PK)
- id_client (FK → CLIENT, NULLABLE)
- telephone_appelant
- date_appel
- duree_appel (secondes)
- transcription (TEXT)
- donnees_collectees (JSON)
- id_reservation_creee (FK → RESERVATION, NULLABLE)
- statut (ENUM: 'complete', 'incomplete', 'erreur')

8. **NOTIFICATION_SMS**

- id_notification (PK)
- id_reservation (FK → RESERVATION)
- telephone_destinataire
- message
- type (ENUM: 'confirmation', 'rappel', 'annulation', 'proposition')
- statut_envoi (ENUM: 'en_attente', 'envoyé', 'echec')
- date_envoi
- id_externe_twilio

7.3 Références Réglementaires

RGPD

- [Texte officiel RGPD](#)
- [Guide du développeur CNIL](#)

Sécurité

- [OWASP Top 10](#)
- [Symfony Security Best Practices](#)

7.4 Ressources Techniques

Documentation APIs

- [Twilio Voice API](#)
- [Twilio SMS API](#)
- [OpenAI API](#)

Frameworks

- [Symfony Documentation](#)
- [React Documentation](#)
- [API Platform](#)

7.5 Outils de Développement

Catégorie	Outil	Usage
IDE	PhpStorm / VS Code	Développement PHP/TypeScript
Maquettes	Figma	Design UI/UX
Diagrammes	Draw.io / PlantUML	UML, MCD/MLD
BDD	MySQL Workbench / pgAdmin	Modélisation et gestion BDD
API Testing	Postman	Tests d'API REST
Versioning	Git + GitHub	Contrôle de version
CI/CD	GitHub Actions	Automatisation tests/déploiement